

Uber

Functional requiements:

user should be able to book a cab based on his preference ,
user should be able to search for locations based on his preference,
rider should be able to accept or reject a ride based on his preference

Non-functional requirements:

Data-partitioning,
load balancing.,
fault tolerance,
vertical scalability,
horizontal scalability,
low latency,
high throughput,

CAP theorem (We need to prioritise consistency for booking scenario in order to avoid double booking or double payment , & for search scenario we need to prioritise 99.99% uptime .i.e Avualability>>Consisterncy)

Core Entities:

user ,
ride,
payment ,
history

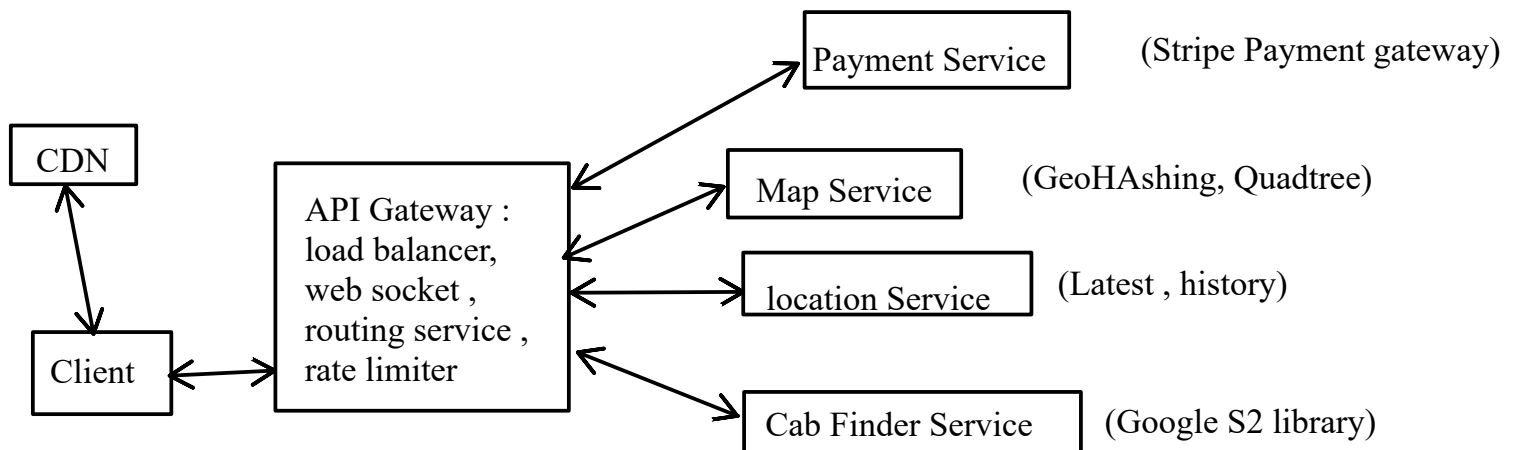
API:

POST:/booking

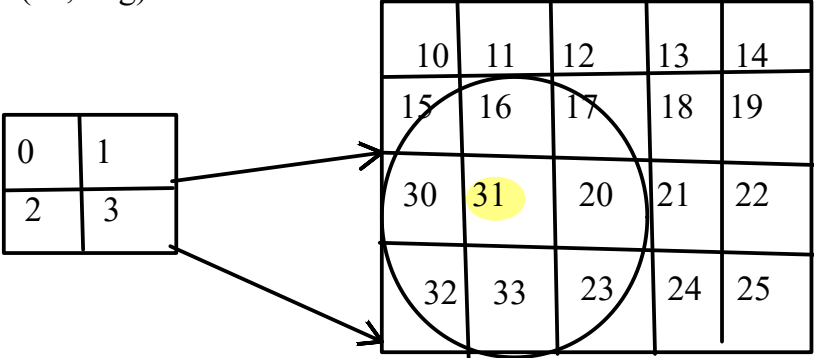
```
{  
  userID,  
  bookingStatus  
}
```

GET:search/{locationID}

HLD :



Map Service
(lat,long) -> chunkID



Cab Finder Service
chunkID -> (riders)



(Google S2 library)

consistent hashing,
gossip protocol